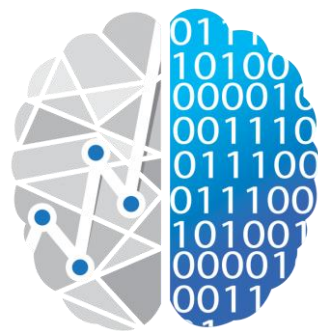




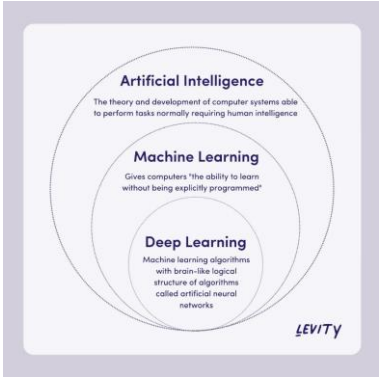
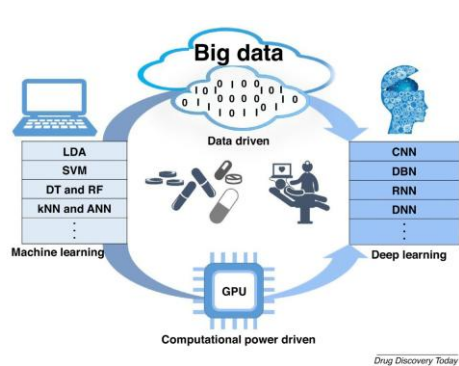
# Machine Learning Intro

Andreas Busjahn

1



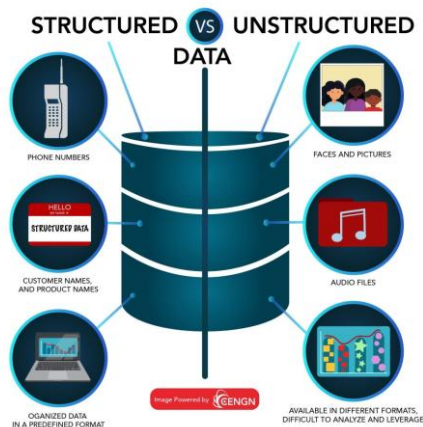
## Different views on concepts



2



## Different data require different methods



3



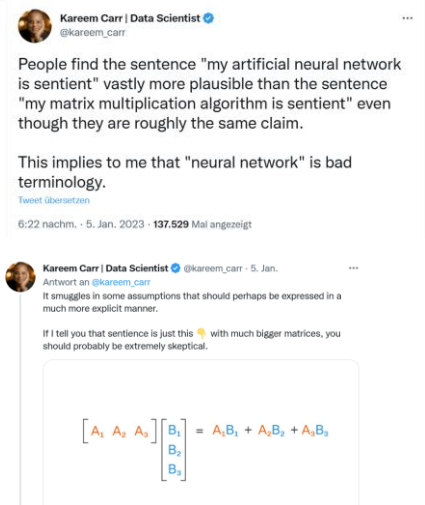
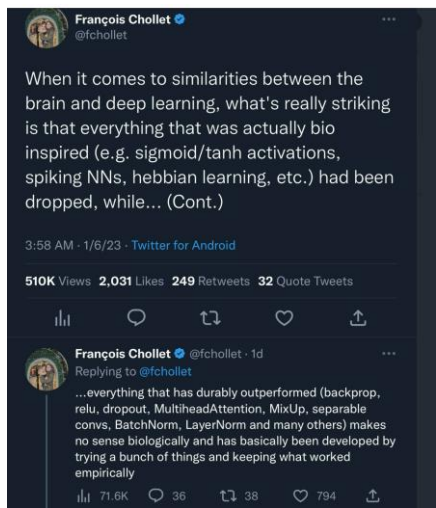
## Machine mimics human?

- Machine Learning is a sub-category of AI, and Deep Learning is a sub-category of ML, meaning they are both forms of AI.
- AI is the broad idea that machines can intelligently execute tasks by mimicking human behaviors and thought processes.
- ML revolves around the idea that machines can learn and adapt through experiences and (structured) data to complete specific tasks.
- DL is a subset of ML. DL models are based on highly complex neural networks that mimic how the brain works. With many layers of processing units, deep learning takes it a step further to learn complex patterns in large amounts of (unstructured) data.

4



# Machine learning is basically brute force



5



# Supervised ML

- Supervised learning is defined by its use of labeled datasets. These datasets are designed to train or “supervise” algorithms into classifying data or predicting outcomes accurately. Using labeled inputs and outputs, the model can measure its accuracy and learn over time.
- Supervised learning can be separated into two types of problems when data mining: classification and regression
  - Linear classifiers, support vector machines, decision trees , logistic regression, and random forest are all common types of classification algorithms.
  - Regression algorithms are linear regression, random forest, and polynomial regression.

6



## Unsupervised ML

- Unsupervised learning uses machine learning algorithms to analyze and cluster unlabeled data sets. These algorithms discover hidden patterns in data without the need for human intervention.
- Unsupervised learning models are used for three main tasks: clustering, association and dimensionality reduction.
  - Clustering is a data mining technique for grouping unlabeled data based on their similarities or differences.
  - Association is another type of unsupervised learning method that uses different rules to find relationships between variables in a given dataset.
  - Dimensionality reduction is a learning technique used when the number of features (or dimensions) in a given dataset is too high.

7



## Computational Complexity and the Big-O Notation

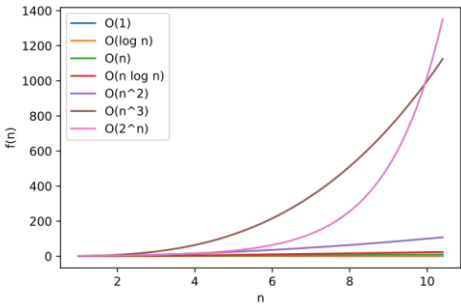
- The Big-O notation is used in both mathematics and computer science to study the asymptotic behavior of functions, i.e., the asymptotic upper bounds.
- In the context of algorithms in computer science, the Big-O notation is most commonly used to measure the time complexity or runtime of an algorithm for the worst-case scenario.
- Often, it is also used to measure memory requirements.
- For e.g. kNN, type of algorithm and number of dimensions are relevant

8



# Runtime as related to complexity

| f(n)           | Name                    |
|----------------|-------------------------|
| 1              | Constant                |
| log n          | Logarithmic             |
| n              | Linear                  |
| n log n        | Log Linear              |
| n <sup>2</sup> | Quadratic               |
| n <sup>3</sup> | Cubic                   |
| n <sup>c</sup> | Higher-level polynomial |
| 2 <sup>n</sup> | Exponential             |



9



# Resampling

- Creation of new samples based on one observed sample.
- Helps in creating new synthetic datasets for training machine learning models and to estimate properties of a dataset when the dataset is unknown, difficult to estimate, or when the sample size of the dataset is small.
- Resampling Methods
  - Permutation tests (also re-randomization tests)
  - Bootstrapping
  - Jackknife
  - Cross validation

10



# Permutation tests

- Permutation reshuffles the observed cases, sampling without replacement
- Random combinations of the 2 classes
- Original / Permutation1

| Class 1  | Class 2 | Class 1  | Class 2 |
|----------|---------|----------|---------|
| Affected | Exposed | Affected | -       |
| Affected | Exposed | Affected | Exposed |
| Affected | -       | Affected | -       |
| Control  | -       | Control  | Exposed |
| Control  | -       | Control  | -       |

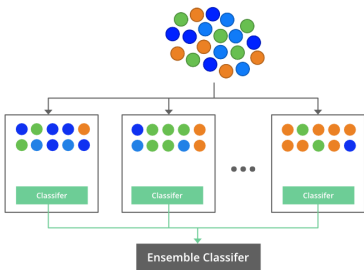
- Repeated x times

11



# Bootstrapping

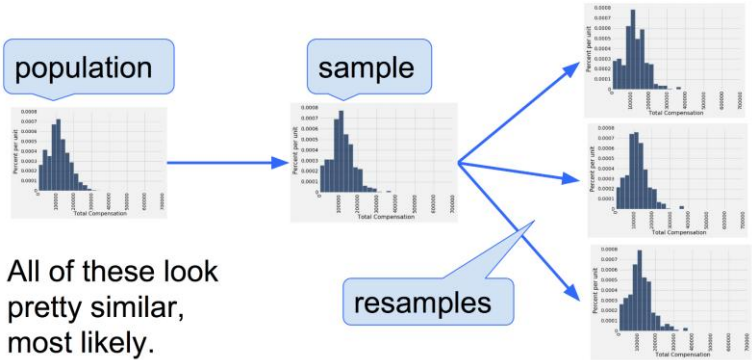
- Bootstrapping selects from the population of observed cases, sampling with replacement.
- Bootstrap is a powerful statistical tool used to quantify the uncertainty of a given model.



12



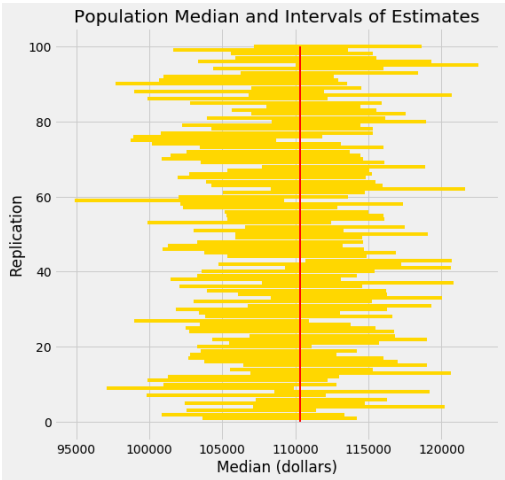
# Bootstrap samples



13



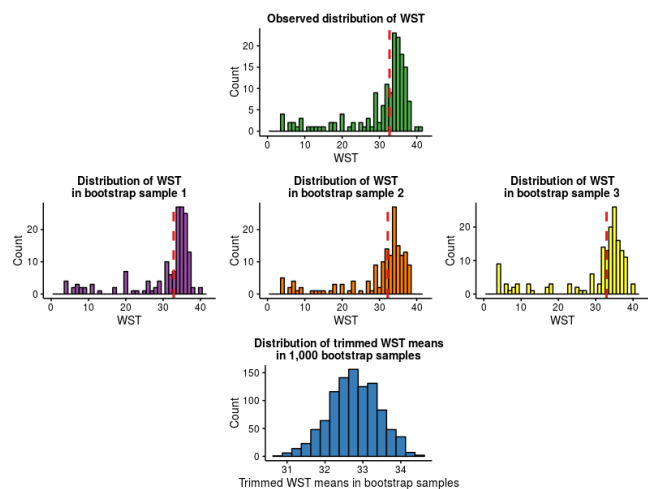
# Bootstrap estimates



14



# Bootstrap Intervals / CIs

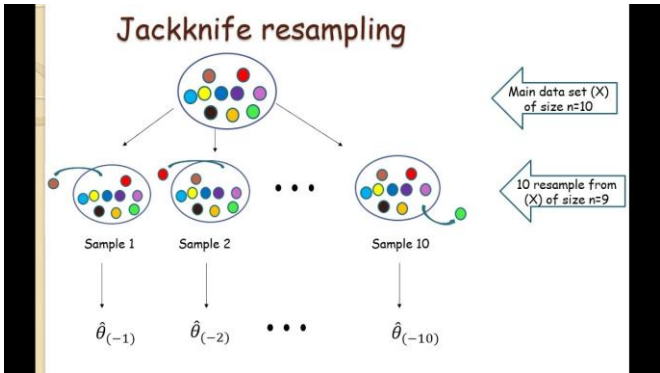


15



# Jackknife

- Jackknife is resampling without replacement
- Creates n samples of size n-1



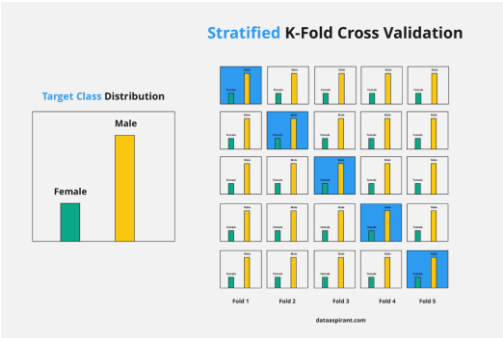
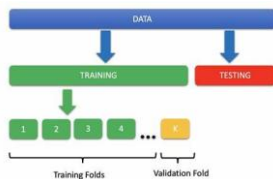
16





# Cross validation

- Splitting sample into k groups
- One group for testing, k-1 groups for training
- Repeated k times, no resampling



17



# Data Preprocessing

- **What is Data Preprocessing?**
- Transforming raw data into clean, usable data for machine learning models.
- Essential for model accuracy and performance.
- **Key Steps in Data Preprocessing:**
- **Data Cleaning:**
- Handling missing values (imputation, deletion).
- Removing noise and inconsistencies.
- **Data Transformation:**
- Scaling numerical features (standardization, normalization).
- Encoding categorical variables (one-hot encoding, label encoding).
- **Data Reduction:**
- Feature selection (choosing relevant features).
- Dimensionality reduction (PCA).

18



## Data Preprocessing

- **Why is Data Preprocessing Important?**
  - Real-world data is often messy and incomplete.
  - Improves model accuracy and efficiency.
  - Prevents bias and overfitting.
- **Challenges in Data Preprocessing:**
  - Choosing the right techniques for the data and model.
  - Handling large datasets efficiently.
  - Avoiding data leakage.
- **Best Practices:**
  - Understand your data.
  - Document your preprocessing steps.
  - Use pipelines to automate preprocessing.

19



## Center/Scale (Standardization)

- **How it works:**
  - Transforms data to have zero mean and unit variance.
- **When to use:**
  - Your data has a Gaussian (normal) distribution, or you expect your model to perform better with normally distributed data. In this case, a transformation like Yeo-Johnson should be done first.
  - You want to preserve the relative relationships between data points while removing the influence of different scales.
  - Your data contains outliers, but they are informative, and you don't want to completely remove their influence. Standardization is less sensitive to outliers than min-max scaling.
  - Algorithms like Support Vector Machines (SVM) and Principal Component Analysis (PCA) are sensitive to feature scaling and often benefit from standardization.

20



## Range (Min-Max Normalization)

- **How it works:**
  - Scales data to a fixed range, usually between 0 and 1.
- **When to use:**
- You need values within a specific range.
  - Your data does not follow a Gaussian distribution.
  - You have a limited number of outliers and they are not critical to your analysis. Min-max scaling is sensitive to outliers, so extreme values can compress the majority of the data into a very small range.
  - Algorithms like neural networks often benefit from having input features scaled to a specific range.

21



## Quantile Normalization

- **How it works:**
- Transforms data to have the same distribution. It involves ranking the data and then replacing the values with quantiles from a standard distribution.
- **When to use:**
- You want to make different distributions comparable.
- Your data has significantly different distributions or contains outliers that heavily influence the other normalization methods.
- You are working with high-dimensional data, such as gene expression data from microarrays or RNA-Seq.
- You want to remove technical variation or batch effects in your data.

22



## Data Preprocessing

- **“center”**: subtract mean from values.
- **“scale”**: divide values by standard deviation.
- Center & scale -> standardize
- **“range”**: normalize values to range [0,1].
- **“BoxCox”**: apply a Box-Cox transform, values must be non-zero and positive.
- **“YeoJohnson”**: apply a Yeo-Johnson transform, like a BoxCox, but values can be negative.
- **“expoTrans”**: apply a power transform like BoxCox and YeoJohnson.
- **“zv”**: remove attributes with a zero variance (all the same value).
- **“nzv”**: remove attributes with a near zero variance (close to the same value).
- **“pca”**: transform data to the principal components. (*ML in itself*)
- **“ica”**: transform data to the independent components. (*ML in itself*)
- **“One-hot encoding”**: Factor levels into columns with 0/1

23



## Power Transformations

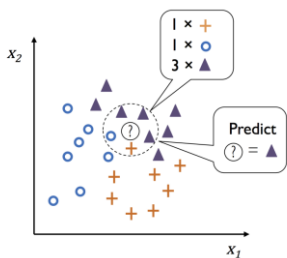
- The Box-Cox transformation is a mathematical tool used in regression analysis and time series analysis to stabilize variance. The transformation is defined as a power function, where the power parameter  $\lambda$  is determined such that the data is closer to a normal distribution. The Box-Cox transformation requires input data to be strictly positive.
- The Yeo-Johnson transformation is an extension of the Box-Cox transformation. It allows for both positive and negative values of the input data. The Yeo-Johnson transformation works similarly to the Box-Cox transformation by inflating low variance data and deflating high variance data to create a more uniform dataset.

24



# kNN Algorithm: Basic Idea

- We have several measures for an object/subject
- We want to classify this into a defined set of groups
- Find the k (e.g. 5) nearest neighbors in a multidimensional space and check their group

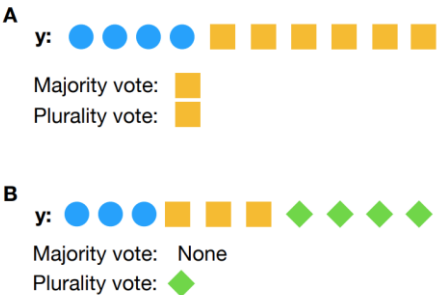


25



# Classification by voting

- K neighbors are contacted and vote for a class
- The plurality vote decides (not majority vote, although it is often called that)



26



## Compute KNN: defining k

- The k value defines how many neighbors to check
- If  $k=1$ , the instance will be assigned to the same class as its single nearest neighbor.
- Defining k can be a balancing act as different values can lead to overfitting or underfitting.
- Lower values of k can have high variance, but low bias, and larger values of k may lead to high bias and lower variance.
- Choice of k will largely depend on the input data as data with more outliers or noise will likely perform better with higher values of k.
- Overall, it is recommended to have an odd number for k to avoid ties in classification
- Cross-validation tactics can help you choose the optimal k for your dataset.

27



## Regression Trees

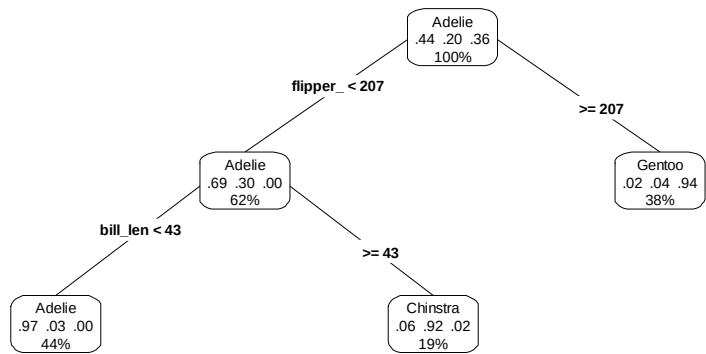
- <https://cran.r-project.org/web/packages/rpart/vignettes/longintro.pdf>
- Classification or regression models of a very general structure using a two stage procedure:
  - Building a hierarchy of binary splits, finding the optimal cut-of definition
  - Pruning of resulting tree to reduce number of nodes
- Importance of predictor variables: Gini index
  - Gini Index, also known as Gini impurity, **calculates the amount of probability of a specific feature that is classified incorrectly when selected randomly**. If all the elements are linked with a single class then it can be called pure
  - Gini Index, like the properties of entropy, **varies between values 0 and 1**, where 0 expresses the purity of classification, i.e. all elements belong to a specified class or only one class exists there. And 1 indicates a random distribution of elements across various classes

28



# Example

- Prediction of penguin species by flipper length and bill length



29



# Random Forrest (of regression trees)

- “Forrest” developed by aggregating trees
- Can be used for classification and regression
- One of the major advantages: it avoids overfitting
- Can deal with a large number of features and helps to identify the important attributes
- The random forest contains two parameters *ntree* and *mtry*
  - *ntree* by default is 500 trees
  - *mtry*- variables randomly sampled as candidates at each split,  $\sqrt{\text{number of features}}$  is a heuristic starting point
- Random Forest Steps
  - Draw *ntree* bootstrap samples.
  - For each bootstrap, grow an un-pruned tree based on a random sample of *mtry* random predictors
  - Predict new data using majority votes for classification and average for regression based on *ntree* trees.

30



## XGBoost

- XGBoost, short for eXtreme Gradient Boosting, is an efficient and scalable implementation of the gradient boosting framework.
- It includes two solvers: a linear model and a tree learning algorithm.
- XGBoost supports various objective functions, including regression, classification, and ranking.
- It has been used to win several Kaggle competitions.

31



## XGBoost

- In a nutshell, gradient boosting is comprised of only three elements:
  - Weak Learners – simple decision trees that are constructed based on purity scores (e.g., Gini).
  - Loss Function – a differentiable function you want to minimize. In regression, this could be a mean squared error, and in classification, it could be log loss.
  - Additive Models – additional trees are added where needed, and a functional gradient descent procedure is used to minimize the loss when adding trees.

32





## Differences between Random Forests and XGBoost

- **Algorithm**
  - RF: bagging technique where an average of subsets is calculated.
  - XGBoost: Each tree is trained to correct errors made by previous.
- **Training Data**
  - RF: selects random subsamples where growing happens in parallel.
  - XGBoost trains specifically the gradient boost data and gradient boost decision trees.
- **Overfitting**
  - RF: Overfitting is reduced with the combination of several underfitting features.
  - XGBoost: overfitting is reduced with the help of regularization parameters to select features based on weak and strong features.
- XGBoost handles **missing data** well along with cross-validation.
- RF is more interpretable as it produces a set of decision trees which can be visualized.

33



## Principal Components Analysis

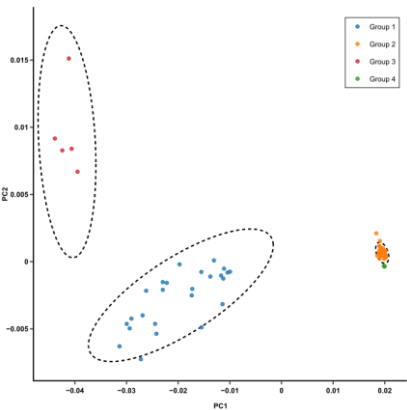
- <https://www.bioconductor.org/packages/release/bioc/vignettes/PCAtools/inst/doc/PCAtools.html>
- Principal Component Analysis (PCA) is a technique for exploratory data analysis
- Allows to better visualize the variation present in a dataset with many variables
- PCA is a way to bring out strong patterns from large and complex datasets
- For example, if you measure the expression of 15 genes from 60 mice, and the data come back as a 15×60 table, how do you make sense of all that? How do you even know which mice are similar to one another, and which ones are different? How do you know which genes are responsible for such similarities or differences?

34



# PCA2

- PCA might be what you need to untangle that data mess.
- It takes the expression data of 15 genes from each mouse and smoosh them down to one single dot that represents the expression profile of that mouse. One dot for one mouse. Sixty mice, sixty dots. The result will look like this:

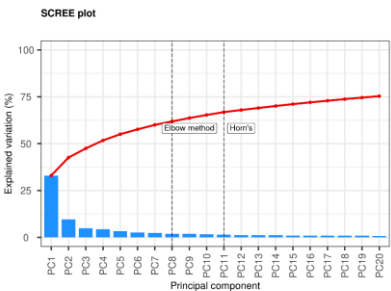


35



# PCA 3

- If the first two or three PCs have capture most of the information, then we can ignore the rest without losing anything important. A scree plot shows how much variation each PC captures from the data. The y axis is eigenvalues, which essentially stand for the amount of variation.
- An ideal curve should be steep, then bends at an “elbow” — cutting-off point — and after that flattens out.
- To deal with a not-so-ideal scree plot curve, there are a couple ways:
  1. Kaiser rule: pick PCs with eigenvalues of at least 1.
  2. Proportion of variance plot: the selected PCs should be able to describe at least 80% of the variance.



36



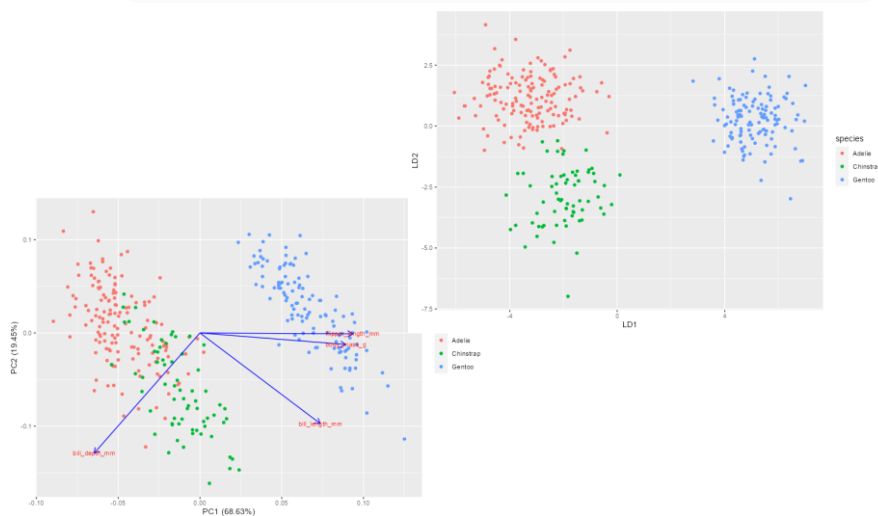
# Linear Discriminant Analysis

- LDA helps to find the linear combination of original variables that provide the best possible separation between the groups. LDA is used e.g. as a preprocessing step in Machine Learning and pattern classification applications
- While PCA optimizes variance explained, LDA optimizes separability
  - The first step is to calculate the separability between different classes(i.e the distance between the mean of different classes) also called as between-class variance
  - Second Step is to calculate the distance between the mean and sample of each class, which is called the within class variance
  - The third step is to construct a lower dimensional space which maximizes the between class variance and minimizes the within class variance.

37



# LDA vs. PCA

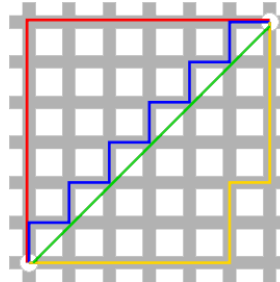
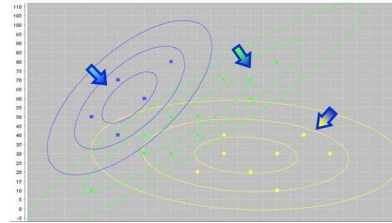


38



## Cluster analysis

- MANY different approaches
  - Exclusive or hierarchical (either bottom-up or top-down)
  - With/without predefined # of clusters
- Various measures of difference
  - Euclidian (as the crow flies, 2-norm, 8.5)
  - Manhattan (taxicab, 1-norm, 12)
  - Mahalanobis: takes into account the correlations of the data set and is scale-invariant, i.e. not dependent on the scale of measurements



39



## Number of clusters

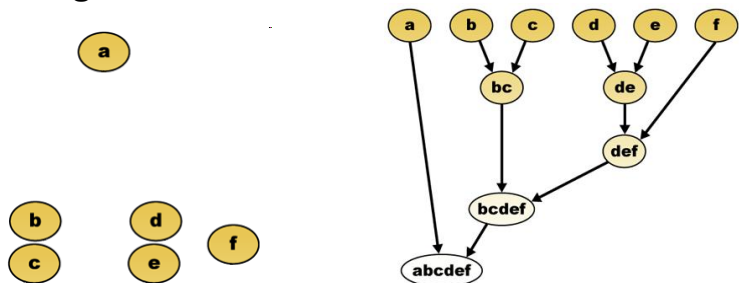
- Based on prior knowledge
- Based on rules of thumb:
  - $k \approx (n/2)^{1/2}$  ( $n=100$ :  $k \approx 7$ ;  $n=500$   $k \approx 16$ ,  $n=500.000$   $k \approx 500$ )
  - Based on percentage of variance explained as a function of the number of clusters: Choose a number of clusters so that adding another cluster doesn't give much better modeling of the data
  - Akaike's information criterion (AIC) is a measure of the goodness of fit of an estimated statistical model, grounded in the concept of entropy, describes the tradeoff between precision and complexity of the model
  - Bayesian (or Schwarz) information criterion (BIC / SIC)

40



# Cluster analysis: Logic

- Start with the most different elements and add all close elements (top-down) or groups of 1..2 close elements (bottom-up)
- a / bc / de / f
- Merge nearest cluster...



41



# K-means clustering

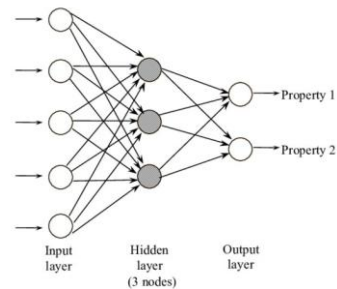
- The K-means algorithm assigns each point to the cluster whose center (also called centroid) is nearest. The center is the average of all the points in the cluster — that is, its coordinates are the arithmetic mean for each dimension separately over all the points in the cluster.
- The algorithm steps are:
  - Choose the number of clusters, k.
  - Randomly generate k clusters and determine the cluster centers, or directly generate k random points as cluster centers.
  - Assign each point to the nearest cluster center.
  - Recompute the new cluster centers.
  - Repeat the two previous steps until some convergence criterion is met (usually that the assignment hasn't changed).
- The main advantages of this algorithm are its simplicity and speed which allows it to run on large datasets.
- Its disadvantage is that it does not yield the same result with each run, since the resulting clusters depend on the initial random assignments.
- It minimizes intra-cluster variance but does not ensure that the result has a global minimum of variance.

42



# Artificial Neural Network

- ANN is a machine learning model inspired by the human brain. It is a system of interconnected nodes that learn to perform tasks by adjusting the weights of the connections between the nodes.
- ANNs can be used for image classification, speech recognition, and natural language processing.



43



## ANN 2

- Each node receives inputs from the nodes in the previous layer and then computes an output based on those inputs. The outputs of the nodes in the final layer are the predictions of the ANN. The weights between the nodes are adjusted during the learning process so that the ANN can make accurate predictions.
- One method of improving: backpropagation

$$\Delta w_{ik}(n) = -\varepsilon^* \delta E_p / \delta w_{ik} + \alpha^* \Delta w_{ik}(n-1)$$

44



## ANNs in R

- Several R packages provide support for ANNs:
  - **neuralnet**: This is a popular package that provides a simple and easy-to-use interface for creating and training ANNs.
  - **nnet**: This package is another popular option that provides more flexibility and control over the ANN architecture.
  - **keras**: This is a high-level R package that provides an interface to the Keras deep learning library.
  - **tensorflow**: This is a popular deep learning library that can also be used in R.

```
library(neuralnet)
# Create the ANN model
model <- neuralnet( formula = y ~ x1 + x2, data = data,
  hidden = c(5, 3), act.fct = "logistic" )
# Train the ANN model
model$fit(data$x, data$y)
# Make predictions on new data
predictions <- predict(model, newdata)
```

45



## Caret: Classification And REgression Training

- package contains tools for:
  - data splitting
  - pre-processing
  - feature selection
  - model tuning using resampling
  - variable importance estimation
- <https://topepo.github.io/caret/index.html>

46



## Methods in caret

- ada, AdaBag, AdaBoost.M1, adaboost, amda, ANFIS, avNNet, awnb, awtan, bag, bagEarth, bagEarthGCV, bagFDA, bagFDAGCV, bam, bartMachine, bayesglm, binda, blackboost, blasso, blassoAveraged, bridge, brnn, BstLm, bstSm, bstTree, C5.0, C5.0Cost, C5.0Rules, C5.0Tree, cforest, chaid, CSimca, ctree, ctree2, cubist, dda, deepboost, DENFIS, dnn, dwdLinear, dwdPoly, dwdRadial, earth, elm, enet, evtree, extraTrees, fda, FH.GBML, FIR.DM, foba, FRBCS.CHI, FRBCS.W, FS.HGD, gam, gamboost, gamLoess, gamSpline, gaussprLinear, gaussprPoly, gaussprRadial, gbm\_h2o, gbm, gcvEarth, GFS.FR.MOGUL, GFS.LT.RS, GFS.THRIFT, glm.nb, **glm**, glmboost, glmnet\_h2o, glmnet, glmStepAIC, gppls, hda, hdda, hdrda, HYFIS, icr, J48, JRip, kernelpls, kknn, **knf**, krlsPoly, krlsRadial, lars, lars2, lasso, **lda**, lda2, leapBackward, leapForward, leapSeq, Linda, **lm**, lmStepAIC, LMT, loclda, logicBag, LogitBoost, logreg, lssvmLinear, lssvmPoly, lssvmRadial, lvq, M5, M5Rules, manb, mda, Mlda, mlp, mlpKerasDecay, mlpKerasDecayCost, mlpKerasDropout, mlpKerasDropoutCost, mlpML, mlpSGD, mlpWeightDecay, mlpWeightDecayML, monmlp, msenet, multinom, mxnet, mxnetAdam, **naive\_bayes**, nb, nbDiscrete, nbSearch, **neuralnet**, nnet, nnls, nodeHarvest, null, OneR, ordinalNet, ordinalRF, ORFlog, ORFpls, ORFridge, ORFsvm, ownn, pam, parRF, **PART**, partDSA, pcaNNet, pcr, pda, pda2, penalized, PenalizedLDA, plr, pls, plsRglm, polr, ppr, pre, PRIM, protocolass, qda, QdaCov, qrf, qrmn, randomGLM, ranger, rbf, rbfDDA, Rborist, rda, regLogistic, relaxo, rf, rFems, RFlda, rfRules, ridge, rlda, rlm, rmda, rocc, rotationForest, rotationForestCp, **rpart**, rpart1SE, rpart2, rpartCost, rpartScore, rqlasso, rqnc, RRF, RRFglobal, rlda, RSimca, rvmLinear, rvmPoly, rvmRadial, SBC, sda, sdwd, simpls, SLAVE, slda, smda, snn, sparseLDA, spikeslab, spls, stepLDA, stepQDA, superpc, **svm**BoundrangeString, svmExpoString, svmLinear, svmLinear2, svmLinear3, svmLinearWeights, svmLinearWeights2, svmPoly, svmRadial, svmRadialCost, svmRadialSigma, svmRadialWeights, svmSpectrumString, tan, tanSearch, treebag, vbmpRadial, vglmAdjCat, vglmContRatio, vglmCumulative, widekernelpls, WM, wsrf, xgbDART, xgbLinear, **xgbTree**, xyf